22. Layered & Clean Architecture: Separation of Concerns in Practice

The Definitive Guide to Decoupling Controllers, Services, and Repositories
Module 4 • Difficulty Level: Intermediate / Advanced • Estimated Reading Time: 13 min

---

🎙️ NotebookLM & Audio Overview Podcast Briefing

Podcast Discussion Hook:
"Why is writing raw database queries inside an HTTP Controller considered a cardinal sin in production engineering, and how does layered architecture shield your codebase?"

3 Core Audio Takeaways:

- 📌 Separation of Concerns mandates that each architectural layer is oblivious to concerns outside its boundary. - 📌 Controllers parse HTTP transports; Services enforce domain business rules; Repositories encapsulate data access. - 📌 Explicitly directing an AI model to structure code along these three layers prevents logic leakage and guarantees testability.

📖 1. The Story & Real-World Analogy

Picture a luxury five-star hotel.

Three distinct professionals operate seamlessly: 1. **The Concierge (Controller):** Greets arriving guests, verifies reservations, and accepts requests. They never step into the kitchen to grill steaks. 2. **The Executive Chef (Service):** Prepares exquisite meals adhering to culinary recipes. They never process guest credit card swipes. 3. **The Pantry Inventory Manager (Repository):** Knows the precise shelf location of aged cheeses and cuts of beef; retrieves them upon chef request.

If the concierge abandons the front desk to wash pots and count pantry crates, the hotel descends into chaos.

Writing database queries inside an Express or FastAPI Controller is that concierge washing dishes in the lobby.

🧠 2. The AI Mental Model & Paradigm Shift

The 3-Tier Layered Architecture Flow:

```
[Client (Web / Mobile App)] ──(HTTP JSON)──> [1. CONTROLLER LAYER]
                                                    | (Sanitized DTO)
                                                    ▼
                                             [2. SERVICE LAYER]
                                              (Core Domain Logic)
                                                    | (Persistence Request)
                                                    ▼
                                             [3. REPOSITORY LAYER]
                                                    | (SQL / Prisma Query)
                                                    ▼
                                             [DATABASE (PostgreSQL)]
```

🤖 3. AI Prompt Lab (Production-Ready Templates)

You can directly paste and adapt this prompt template in Cursor, Claude Code, ChatGPT, or Copilot:

```
# ROLE: Clean Architecture Lead
We are building a user registration feature using Node.js, Express, and Prisma.

## TASK:
Architect this feature strictly adhering to 3-tier Layered Architecture:
1. `user.controller.ts`: Handles HTTP req/res, status codes, and Zod parsing
exclusively.
2. `user.service.ts`: Handles password hashing and business validation (never
accepts HTTP req/res!).
3. `user.repository.ts`: Encapsulates pure Prisma database persistence.
```

💻 4. Code Walkthrough: Naive / Hallucination vs. Production-Grade

❌ Naive / Unchecked AI Output (Risky):

```
// Felaket: Controller içinde hem validasyon, hem hashleme, hem SQL!
app.post("/users", async (req, res) => {
  if (!req.body.email) return res.status(400).send("Email gerekli");
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = await prisma.user.create({ data: { email: req.body.email, password:
hashed } });
  await sendEmail(user.email);
  res.json(user); // Şifre hash'i bile frontend'e sızdı!
});
```

✅ Senior-Refined Version (Robust & Clean):

```
// 1. REPOSITORY KATMANI (user.repository.ts)
export class UserRepository {
  async create(data: { email: string; passwordHash: string }) {
    return await prisma.user.create({ data });
  }
  async findByEmail(email: string) {
    return await prisma.user.findUnique({ where: { email } });
  }
}

// 2. SERVICE KATMANI (user.service.ts - Saf İş Mantığı)
export class UserService {
  constructor(private readonly userRepo: UserRepository) {}

  async registerUser(dto: RegisterUserDto): Promise<SafeUser> {
    const existing = await this.userRepo.findByEmail(dto.email);
    if (existing) throw new ConflictError("E-posta zaten kayıtlı.");

    const passwordHash = await hashPassword(dto.password);
    const user = await this.userRepo.create({ email: dto.email, passwordHash });

    return { id: user.id, email: user.email }; // Şifre asla dönmez!
  }
}

// 3. CONTROLLER KATMANI (user.controller.ts - Sadece HTTP Protokolü)
export class UserController {
  constructor(private readonly userService: UserService) {}

  async handleRegister(req: Request, res: Response): Promise<void> {
    const validated = RegisterUserSchema.parse(req.body);
    const user = await this.userService.registerUser(validated);
    res.status(201).json({ success: true, data: user });
  }
}
```

💡 **Code Analysis:** Decoupling layers yields extreme durability: swapping Prisma for raw SQL impacts only the Repository; migrating from Express to Fastify impacts only the Controller. Pure domain logic inside the Service remains untouched.

⚠️ 5. Junior Pitfalls & Senior Perspective

- **Trap:** Passing HTTP `req` and `res` objects into the Service layer.
  - **Remedy:** Services must remain transport-agnostic, consuming only plain Data Transfer Objects (DTOs).
- **Trap:** Leaking database entity models containing password hashes directly to frontend clients.
  - **Remedy:** Always map entities through explicit Response DTOs exposing only safe fields.
- **Trap:** Over-architecting trivial pass-through CRUD endpoints into 10 layers of abstraction.
  - **Remedy:** Remain pragmatic; avoid unnecessary boilerplate when zero business transformation exists.

🎬 6. Udemy Video Lecture Blueprint & Slide Deck

📽️ Lecture Title: *Lecture 22: Layered Architecture - Controller, Service & Repository*

- 🖥️ **Slide 1:** What is Separation of Concerns and why does it protect sanity?
- 🖥️ **Slide 2:** The 3-tier contract: Concierge, Chef, and Pantry Manager.
- 🖥️ **Slide 3:** Keeping Services transport-agnostic for universal reusability.
- 🖥️ **Slide 4:** Live demo: Refactoring a bloated Express route into 3 distinct layers.

Live Coding / Screencast Scenario:
Open an monolithic Express route. Extract data queries into `UserRepository`, logic into `UserService`, and routing into `UserController`. Demonstrate writing an isolated unit test for `UserService` in 60 seconds.

🎯 7. Daily Action Task & Mini Quiz

🛠️ Daily Action Task:

Isolate an inline database query from an API route in your codebase and extract it into a dedicated repository class.

🧠 Chapter Review Quiz:

Question 1: In a 3-tier Layered Architecture, what is the sole mandate of the 'Service' layer? - A) Evaluating HTTP status headers (200, 404) - B) Enforcing business rules, orchestrating domain workflows, and executing calculations - C) Assembling raw SQL query statements - D) Styling CSS UI buttons > **Correct Answer:** B > **Explanation:** The Service layer houses pure business logic, remaining agnostic of delivery protocols and database dialects.

Question 2: Why is passing `(req, res)` objects directly into Service methods an architectural anti-pattern? - A) It impairs compiler speed - B) It couples domain logic tightly to HTTP transports, preventing reuse in CLI scripts, background workers, or unit tests - C) TypeScript automatically rejects it - D) It induces display resolution drops > **Correct Answer:** B > **Explanation:** Services must accept transport-agnostic DTOs so the same business logic can be triggered by background queues or CLI scripts.